Simulation-Based Bayesian Estimation of Dynamic Models

Cameron Fen

February 5th, 2022

Outline

1 Introduction

2 The SNPE Algorithm

3 Normalizing Flows

4 Results

In a Literature as Quantitative as Macro, why are we Calibrating or using Partial Information Techniques? How can we Overcome the Barriers to Bayesian Estimation?

Using the Sequence-Space Jacobian to Solve and Estimate Heterogeneous-Agent Models

Adrien Auclert* Bence Bardóczy† Matthew Rognlie‡ Ludwig Straub§

March 2021

Abstract

We propose a general and highly efficient method for solving and estimating general equilibrium heterogeneous-agent models with aggregate shocks in discrete time. Our approach relies on the rapid computation of *sequence-space Jacobians*—the derivatives of perfect-foresight equilibrium mappings between aggregate sequences around the steady state. Our main contribution is a fast algorithm for calculating Jacobians for a large class of heterogeneous-agent problems. We combine this algorithm with a systematic approach to composing and inverting Jacobians to solve for general equilibrium impulse responses. We obtain a rapid procedure for likelihood-based estimation and computation of nonlinear perfect-foresight transitions. We apply our methods to three canonical heterogeneous-agent models: a neoclassical model, a New Keynesian model with one asset, and a New Keynesian model with two assets.

Main Complaint Against Bayesian and MLE estimation

- I think most people acknowledge that on the margin, there are benefits to Bayesian estimation or at least MLE, but it's just too damn hard:
  1. I don't think most people think it's too hard to code, but mainly focus on the fact that the estimation procedure could takes weeks of computation
  2. Additionally it probably won't work with your favorite solution method
- Thus, most people don't bother

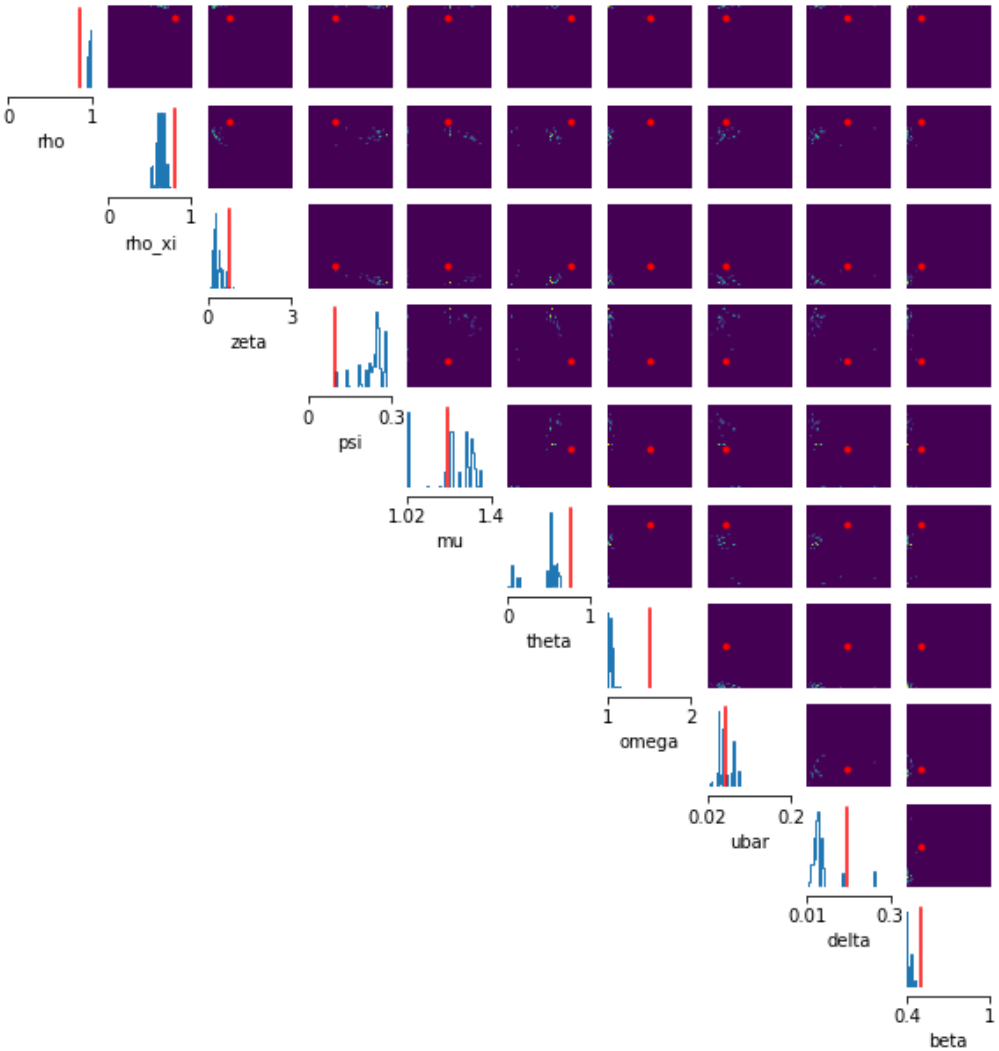
What's so good about Bayesian or MLE estimation?

- Estimation provides a reproducible process, rigor, and the ability to quantify performance of macro economic models
- Bayesian estimation allows one to use priors informed by microdata to add information to the estimation procedure
  - One can also derive distributional information so, for example, one can interpret the likelihood of depreciation rate being almost 0% versus being around 35%
- MLE estimation provides a parameter estimate that is statistically efficient compared to alternative simulation based approaches

Speed

Speed: I spent 520 hours Estimating a HANK model and here is my Posterior

Speed: I spent 520 hours Estimating a HANK model and here is my Posterior



Solution Method Compatibility

Solution Methods: A Brief Review

- Solution methods concern itself with solving for the correct policy function, estimation concerns itself with finding the set of parameters that when solved generate realistic data
- **Projection:** Uses a function approximator for control variables such that the policy function satisfies all the first order conditions of the structural model
- **Perturbation:** Uses a Taylor series approximation around the steady state to estimate a model.
  - Perturbation is the only method that yields a complementary likelihood function when estimated. As such it is the only solution approach conventional methods can Bayesian and MLE estimate
- **Value Function Iteration:** Generates estimates to the value function by using dynamic programming to iterate on the Bellman equation

2. It Doesn't Work with My Favorite Solution Method

Alternative Methods for Solving
Heterogeneous Firm Models

Stephen J. Terry*

April 2017

Abstract

I implement and compare five solution methods for a benchmark heterogeneous firms model with lumpy capital adjustment and aggregate uncertainty. The Krusell Smith algorithm performs best within a group of methods using projection in the aggregate states. Another technique, Parameterization plus Perturbation, is much faster and performs best within a group of methods using perturbation in aggregates. However, projection and perturbation have nonoverlapping strengths and weaknesses. I highlight the resulting tradeoffs with several model extensions. I recommend that researchers apply projection methods to cases with large shocks or nonlinear dynamics, while cases with explicitly distributional channels at work favor perturbation.

Large Literature Attempting to Estimate Dynamic Models with Non-Linearities

- Large literature estimating non-smooth dynamic models
  - **s-S Models**: Arrow et. al. 1951, Caplin and Spulber, 1987, Bertola and Caballero 1990, House and Leahy 2004, Caplin and Leahy 2006, Kahn and Thomas 2008, etc
  - **Structural Finance**: He, Whited, and Guo 2021, Terry, Whited, and Zakolyukina, 2021, Taylor 2012, Whited and Guo 2006 etc.
- There is difficulty in building efficient estimation methods for these models:
  - **Method of Simulated Moments**: McFadden 1989, Pakes and Pollard 1989, Duffie and Singleton 1990, etc.
  - **Efficient MSM, Indirect Inference, and Maximum Simulated Likelihood**: Smith Jr 1993, Gourierou, Monfort, and Renault 1993, Kristensen and Shin 2012, Lerman and Manski 1981, etc.
  - **Machine Learning**: Kaji, Manresa, and Pouliot 2020

Bayesian Basics

Bayes Rule

- Bayes Rule:

$$P(\theta|X) = \frac{P(X|\theta)P(\theta)}{P(X)} \tag{1}$$

- θ are parameters, $P(\theta)$ prior, X data, $P(X|\theta)$ likelihood, $P(\theta|X)$ posterior

The Traditional Approach: Metropolis-Hastings Markov Chain Monte Carlo (MCMC)

- If you assume you know the likelihood function: $P(X|\theta)$, since the prior is known, the numerator of Bayes rule is always known.
  - However the denominator is intractable
- MCMC is concerned with sampling from the posterior, even if the Bayes denominator is not available.
- MCMC involves moving to another point in proportion to the ratio of the unnormalized Bayes numerator
- For example if $P(X|\theta)P(\theta)$ has twice the probability mass as $P(X|\theta')P(\theta')$ then with 50% probability one moves to θ' and 50% probability one revisits the same intial point θ
- This method is quite slow as one has to visit all points in proportion to their likelihood and high likelihood points many times.

A Solution to Both Problems

Simulation Based Infernce

- This paper proposes a deep learning approach to simulation-based inference that performs MLE or Bayesian posterior estimation without a likelihood function, allowing this procedure to work with any solution method
  - This approach is also upwards of 10x faster than traditional Metropolis-Hasting Methods for Bayesian Inference
- The approach separates estimation from solution allowing for specialization so that the macroeconomist can use boilerplate code to perform the estimation out of the box.
- Outside of macro (IO, Applied Theory, Econometrics) this algorithm can estimate models with intractable likelihoods even with a latent state space time series structure

Outline

1 Introduction

2 The SNPE Algorithm

3 Normalizing Flows

4 Results

The Question Statement

- Simulation-based inference concerns itself with a more difficult problem than Metropolis-Hastings: If you do not know the likelihood function, can you still perform Bayesian estimation and derive the optimal posterior
- MLE is an easy extension: One can perform maximum likelihood by taking the mode of the posterior
- One learns the posterior by simulating from the joint distribution then fitting a conditional density estimator on the joint.

Sequential Neural Posterior Estimation (SNPE): The Approach

- This is the approach I use and shares similarities with Kristensen and Shin (2012)

1. Sample $\theta \sim P(\theta)$
2. Sample $X' \sim P(X'|\theta)$ to generate (X', θ) pairs
3. Use a density estimator for the conditional density $P(\theta|X')$
4. Then $P(\theta|X' = X)$ is the posterior

The Kernel Density Estimator

- The the weakness in Kristensen and Shin (2012) is the use of a kernel density estimator (KDE) to estimate the posterior pdf.
- A KDE is a density estimator that estimates a density by taking a weighted average of sample points weighted by how close the points are to the density point in question
- My methodology is the same as Kristensen and Shin (2012), however I use a normalizing flow density estimator versus the KDE they use

The Normalizing Flow Density Estimator

- Unlike a KDE:
  - A normalizing flow density estimator can handle high dimensional data like data simulated from macro models, and...
  - it can estimate conditional densities
- The methodological improvement in my algorithm comes entirely from the advantages that the normalizing flow density estimator has over a kernel density estimator in especially these two areas

Outline

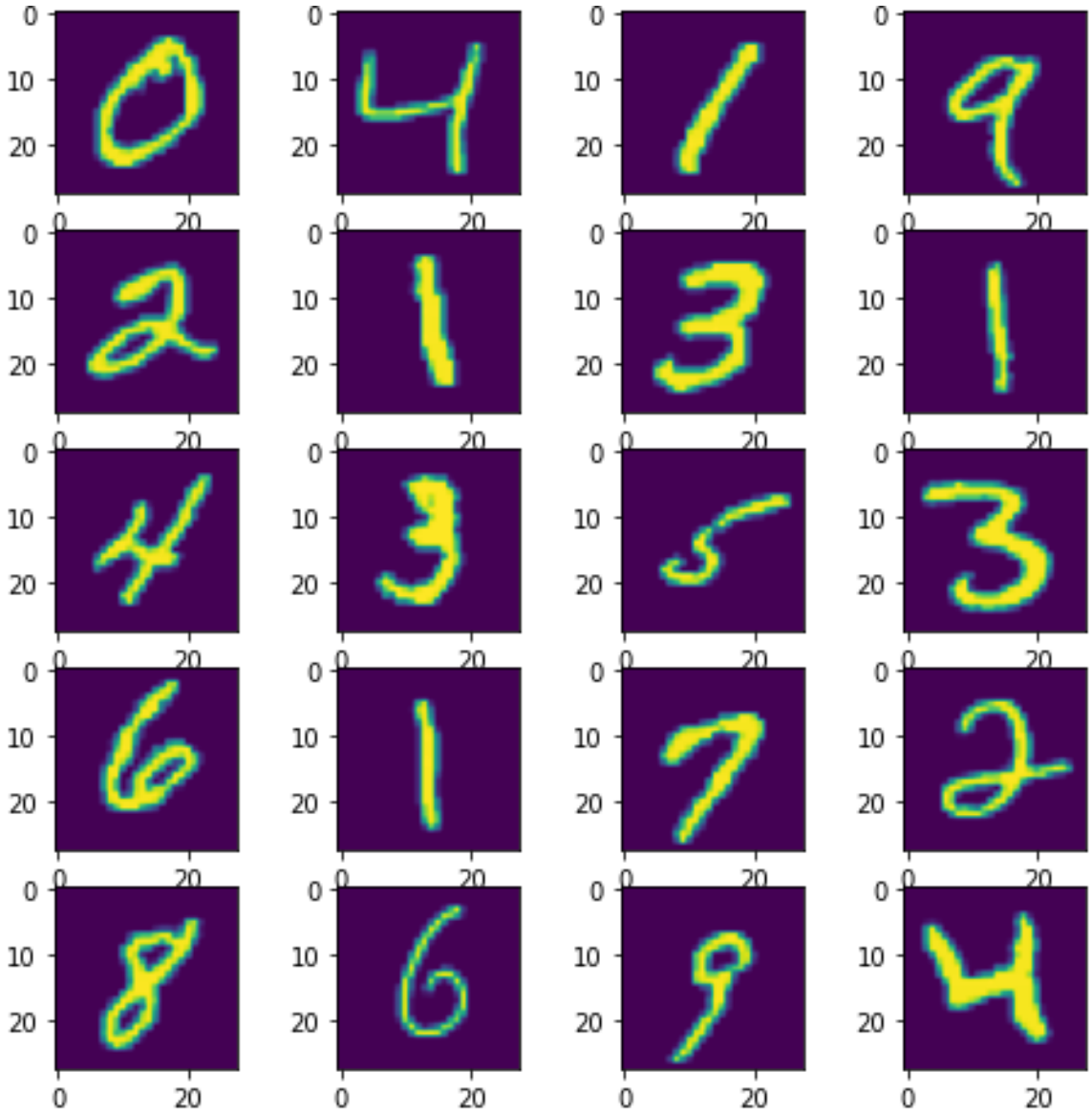
1 Introduction

2 The SNPE Algorithm

3 Normalizing Flows

4 Results

Normalizing Flow Example



The Normalizing Flow (Rezende and Mohamad 2015)

- The normalizing flow is a underused, if not novel, technique in economics
- A normalizing flow chains together stacks of bijectors so that the output of one bijector becomes the input of the next bijector
- The particular bijector I use is the neural autoregressive bijector (Huang et. al. 2018)
- As the name implies a bijector is a bijective function that maps (typically distributions) from $R^n \rightarrow R^n$
- Additionally the bijector is invertable not only in theory but in practice
- The nuts and bolts, the next couple slides, are difficult to explain in a short seminar for more information see: https://blog.evjang.com/2018/01/nf1.html

Bijectors

- Given an N dimensional vector of inputs ranging from $i = 1...n$, the outputs of a neural autoregressive bijector is defined as:

$$y_1 = \sigma(a * z_1 + b) \tag{2}$$

$$y_i = \sigma(\alpha(z_1...z_{i-1})z_i + \beta(z_1...z_{i-1})) \tag{3}$$

- Here $\alpha(.)$ and $\beta(.)$ are feed forward neural networks that only depend on z_i that have a lower index than the output
- You can easily see that you can calculate the forward direction: y from z and the inverse direction: z from y

Bijectors: Discussing Sigma

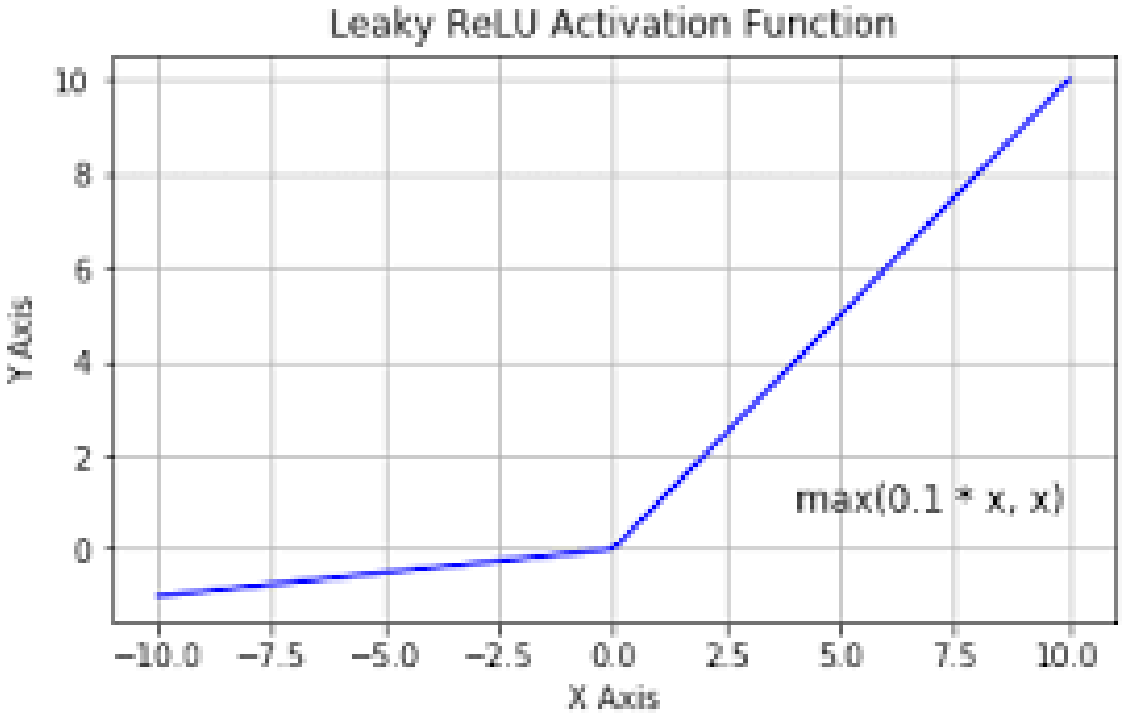
- Given an N dimensional vector of inputs ranging from $i = 1...n$, the outputs of a neural autoregressive bijector is defined as:

$$y_1 = \boldsymbol{\sigma}(a * z_1 + b) \tag{4}$$

$$y_i = \boldsymbol{\sigma}(\alpha(z_1...z_{i-1})z_i + \beta(z_1...z_{i-1})) \tag{5}$$

- Here $\alpha(.)$ and $\beta(.)$ are feed forward neural networks that only depend on z_i that have a lower index than the output
- You can easily see that you can calculate the forward direction: y from z and the inverse direction: z from y

Sigma: The Leaky ReLU



Bijectors: Inverting an Affine Function

- Given an N dimensional vector of inputs ranging from $i = 1...n$, the outputs of a neural autoregressive bijector is defined as:

$$y_1 = \sigma(\boldsymbol{a} * z_1 + \boldsymbol{b}) \tag{6}$$

$$y_i = \sigma(\alpha(z_1...z_{i-1})z_i + \beta(z_1...z_{i-1})) \tag{7}$$

- Here $\alpha(.)$ and $\beta(.)$ are feed forward neural networks that only depend on z_i that have a lower index than the output
- You can see that you can calculate the forward direction: y from z and the inverse direction: z from y

Bijectors: Inverting an Affine Function

- Given an N dimensional vector of inputs ranging from $i = 1...n$, the outputs of a neural autoregressive bijector is defined as:

$$\frac{\sigma^{-1}(y_1) - \boldsymbol{b}}{\boldsymbol{a}} = z_1 \tag{8}$$

$$y_i = \sigma(\alpha(z_1...z_{i-1})z_i + \beta(z_1...z_{i-1})) \tag{9}$$

- Here $\alpha(.)$ and $\beta(.)$ are feed forward neural networks that only depend on z_i that have a lower index than the output
- You can see that you can calculate the forward direction: y from z and the inverse direction: z from y

Bijectors: Inverting a Normalizing Flow

- Given an N dimensional vector of inputs ranging from $i = 1...n$, the outputs of a neural autoregressive bijector is defined as:

$$y_1 = \sigma(\boldsymbol{a} * z_1 + \boldsymbol{b}) \tag{10}$$

$$y_i = \sigma(\boldsymbol{\alpha}(\boldsymbol{z}_1 ... \boldsymbol{z}_{\boldsymbol{i}-1}) z_i + \boldsymbol{\beta}(\boldsymbol{z}_1 ... \boldsymbol{z}_{\boldsymbol{i}-1})) \tag{11}$$

- Here $\alpha(.)$ and $\beta(.)$ are feed forward neural networks that only depend on z_i that have a lower index than the output
- You can see that you can calculate the forward direction: y from z and the inverse direction: z from y

Bijectors: Inverting a Normalizing Flow

- Given an N dimensional vector of inputs ranging from $i = 1...n$, the outputs of a neural autoregressive bijector is defined as:

$$y_1 = \sigma(\boldsymbol{a} * z_1 + \boldsymbol{b}) \tag{12}$$

$$y_i = \sigma(\boldsymbol{\alpha}_{\boldsymbol{z}_{1:i-1}} z_i + \boldsymbol{\beta}_{\boldsymbol{z}_{1:i-1}}) \tag{13}$$

- Here $\alpha(.)$ and $\beta(.)$ are feed forward neural networks that only depend on z_i that have a lower index than the output
- You can see that you can calculate the forward direction: y from z and the inverse direction: z from y

Bijectors: Inverting a Normalizing Flow

- Given an N dimensional vector of inputs ranging from $i = 1...n$, the outputs of a neural autoregressive bijector is defined as:

$$\frac{\sigma^{-1}(y_1) - \boldsymbol{b}}{\boldsymbol{a}} = z_1 \tag{14}$$

$$\frac{\sigma^{-1}(y_i) - \beta_{\boldsymbol{z}_{1:i-1}}}{\boldsymbol{\alpha}_{\boldsymbol{z}_{1:i-1}}} = z_i \tag{15}$$

- Here $\alpha(.)$ and $\beta(.)$ are feed forward neural networks that only depend on z_i that have a lower index than the output
- You can see that you can calculate the forward direction: y from z and the inverse direction: z from y

The Change in Variable

- The flow learns a mapping from a known "base" distribution like a multivariate normal random variable to a "target" distribution one wants to estimate
- One trains the flow by modifying the parameters the model to learn the correct change of variable from the base distribution to the target distribution
- If the base distribution has a pdf of $P(z)$, the output distribution of a flow f has a pdf of $P(y) = P(z)|det(df/dz)|^{-1}$, based on a change of variable formula
  - The Jacobian term follows the same logic as the canonical 1st year econometrics example of a change of variable from a normal to a chi-squared distribution
- Thus you can perform density estimation by maximizing the likelihood of the data samples occurring under the flow

Example: Estimating the Likelihood of a Point

- Let's say we want to estimate the likelihood of a point $y = (1.5, -1.5)$ under a flow
- Procedure:
  1. Use the ability of bijectors to perform inversion to find the z point that would generate $y = (1.5, -1.5)$
  2. Say this point is $z = (0.2, 1.0)$. Then calculate $P(z = (0.2, 1.0))$ since one knows the base distribution by construction
  3. Evaluate the inverse of the determinant of the Jacobian at $z = (0.2, 1.0)$
  4. Derive the likelihood by putting the parts together to get $P(y) = P(z)|det(df/dz)|^{-1}$
  5. Knowing this point, maximize the likelihood, $P(y)$, by taking gradient steps in the direction of steepest increase.

Outline

1 Introduction

2 The SNPE Algorithm

3 Normalizing Flows

4 Results

Models

- For the empirical tests, so far this paper has estimated four models:
  1. A four parameter toy RBC model that is solved using time iteration (Rendahl 2https://www.overleaf.com/project/6154789b4efbe708b6ff206c017) using simulated data as well as real data
  2. A four parameter model of capital formation solved via value function iteration from simulated data
  3. A ten parameter HANK model solved via Rieter's method and time iteration with x that contains 61x861 timestep-covariates from simulated data
  4. A eleven parameter bequests model solved via value function iteration

Models

Models	SBI Estimates	MH-Estimates	Research Scale
RBC Model:	X	X	☐
Cash Flow Model:	X	☐	☐
Lucas Tree Model:	X	☐	☐
HANK Model:	X	X	X
Bequests Model:	X	☐	X

RBC Model Description

- From Alisdair McKay: https://alisdairmckay.com/Notes/HetAgents/index.html
- Standard CRRA utility:

$$u(C_t) = E \sum_t \beta^t \frac{C_t^{1-\gamma}}{1-\gamma} \tag{16}$$

- Cobb-Douglas production, standard law of motion, and productivity AR in logs

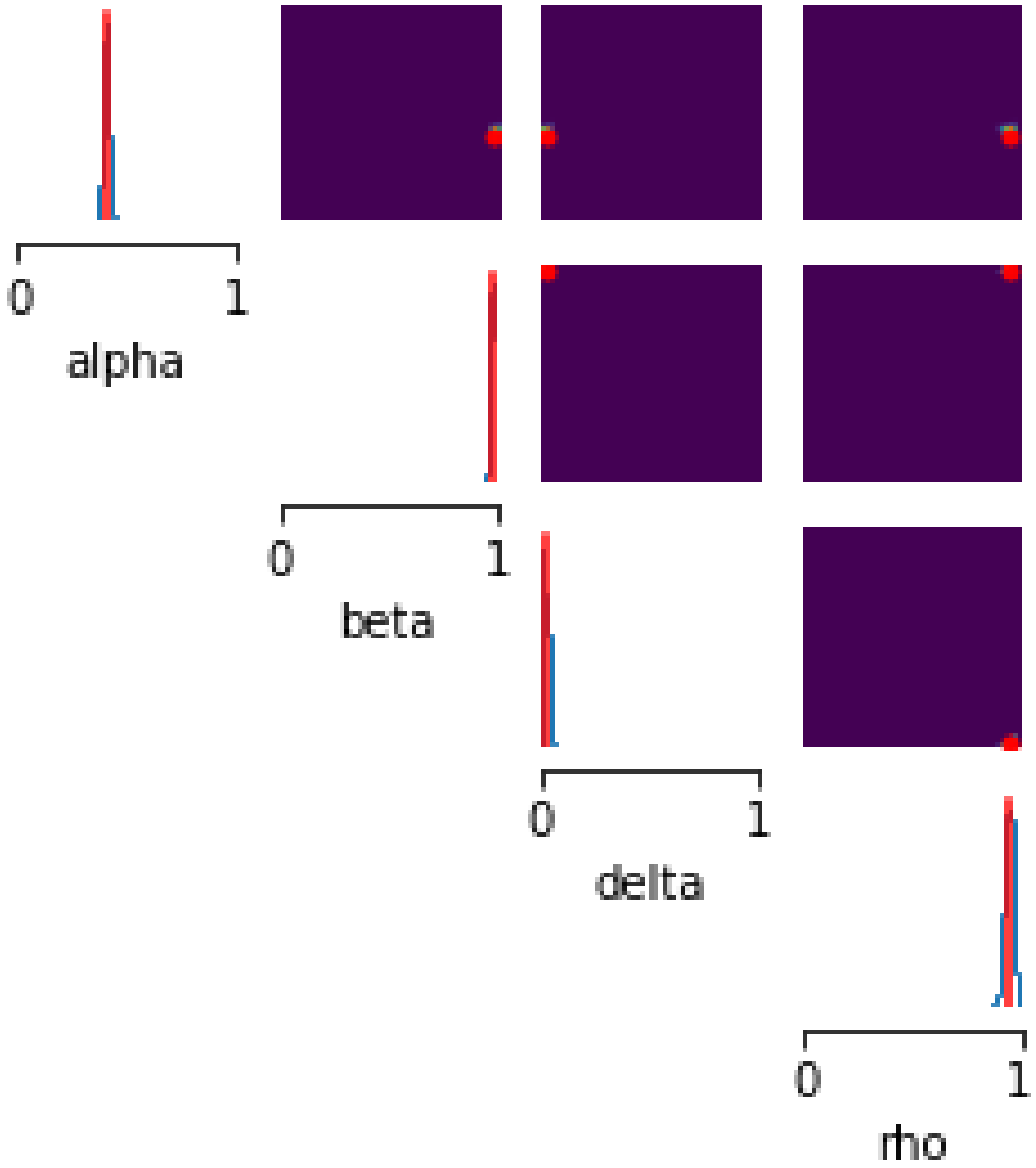
$$Y_t = Z_t K_t^{\alpha} \bar{L}^{1-\alpha} \tag{17}$$

$$K_t = (1-\delta)K_{t-1} + Y_t - C_t \tag{18}$$

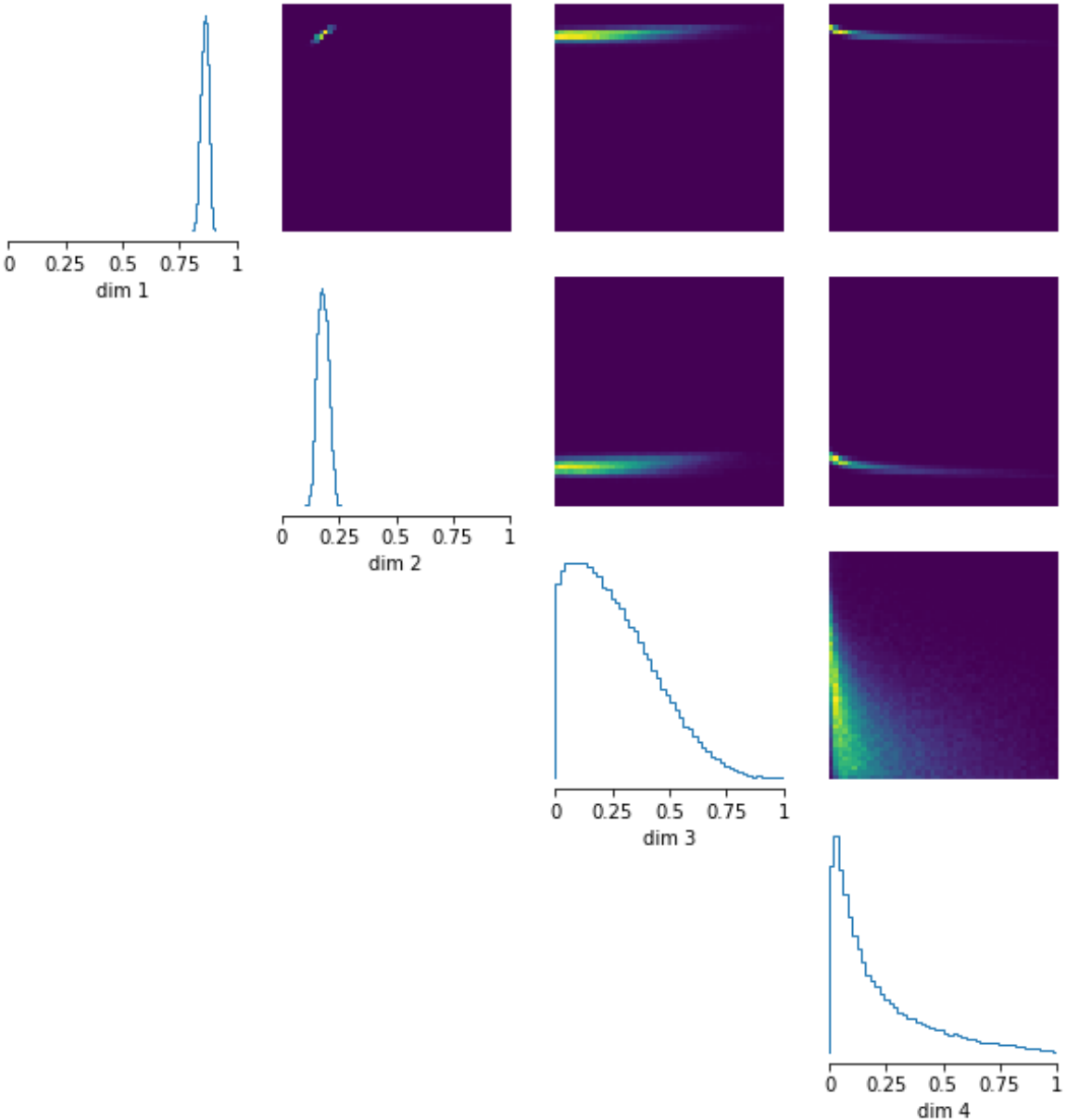
$$logZ_t = \rho logZ_{t-1} + \epsilon_t \tag{19}$$

$$\tag{20}$$

Simulated Data RBC



Real Data RBC



HANK Model: Household + Output

- Again from Alisdair McKay
- Like the RBC model uses CRRA utility

$$u(C_t) = E \sum_t \beta^t \frac{C_t^{1-\gamma}}{1-\gamma} \tag{21}$$

$$\tag{22}$$

- intermediate good output is a function of productivity and labor only and the final good is aggregated via Dixit-Stiglitz

$$a_t + C_t = R_t * a_{t-1} + earnings \tag{23}$$

$$Y_t = A_t \int_0^1 n_{j,t} dj = A_t N_t \tag{24}$$

$$\tag{25}$$

- Output is a function of consumption and search costs

$$Y_t = C_t + \psi M_t H_t \tag{26}$$

HANK Model: Government + Labor

- Government has to balance both nominal and real budget:

$$\tau P_t(w_t + d_t)(1 - u_t) + P_t B = (1 + i_{t-1})P_{t-1}B + P_t b u_t \tag{27}$$

$$\tau(w_t + d_t)(1 - u_t) + B = R_t B + b u_t \tag{28}$$

- Standard McCall Search

$$N_t = (1 - \delta)N_{t-1} + H_t \tag{29}$$

$$1 - u_t = (1 - \delta)(1 - u_{t-1}) + H_t \tag{30}$$

$$M_t = \frac{H_t}{u_{t-1} + \delta N_{t-1}} \tag{31}$$

$$M_t = \frac{1 - u_t - (1 - \delta)(1 - u_{t-1})}{u_{t-1} + \delta(1 - u_{t-1})}. \tag{32}$$

- Earnings has two states: employed with taxes and unemployed with benefits.

HANK Model: Firm

- Firm uses Calvo pricing and the above Dixit-Stiglitz aggregator to derive the New Keynesian price curve.

$$Y_t = \left(\int_0^1 y_{j,t}^{\frac{\epsilon-1}{\epsilon}}\right)^{\frac{\epsilon}{\epsilon-1}} \tag{33}$$

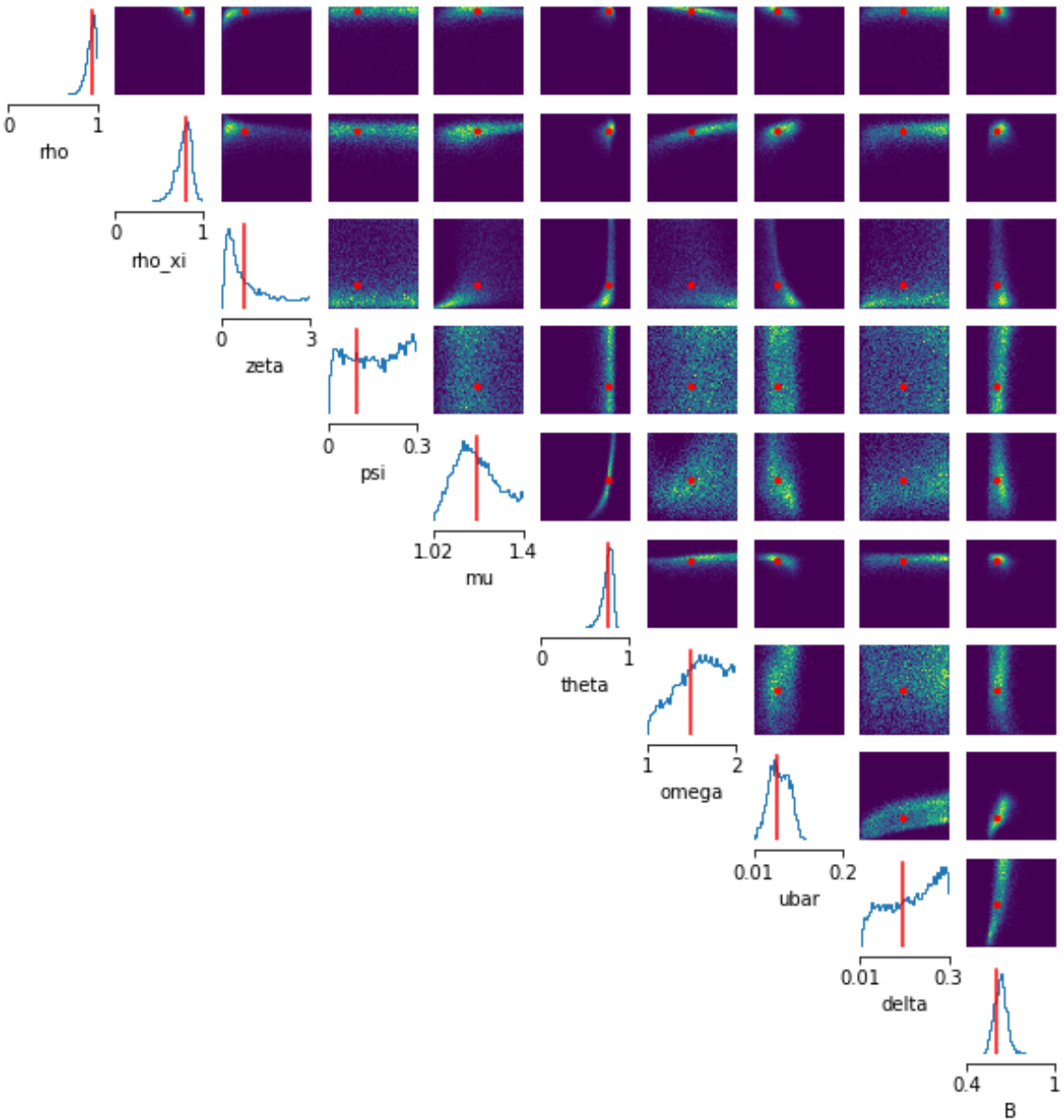
$$y_{j,t} = \left(\frac{p_{j,t}}{P_t}\right)^{-\varepsilon} Y_t \tag{34}$$

$$P_t^{1-\varepsilon} = \int_0^1 p_{j,t}^{1-\varepsilon} dj. \tag{35}$$

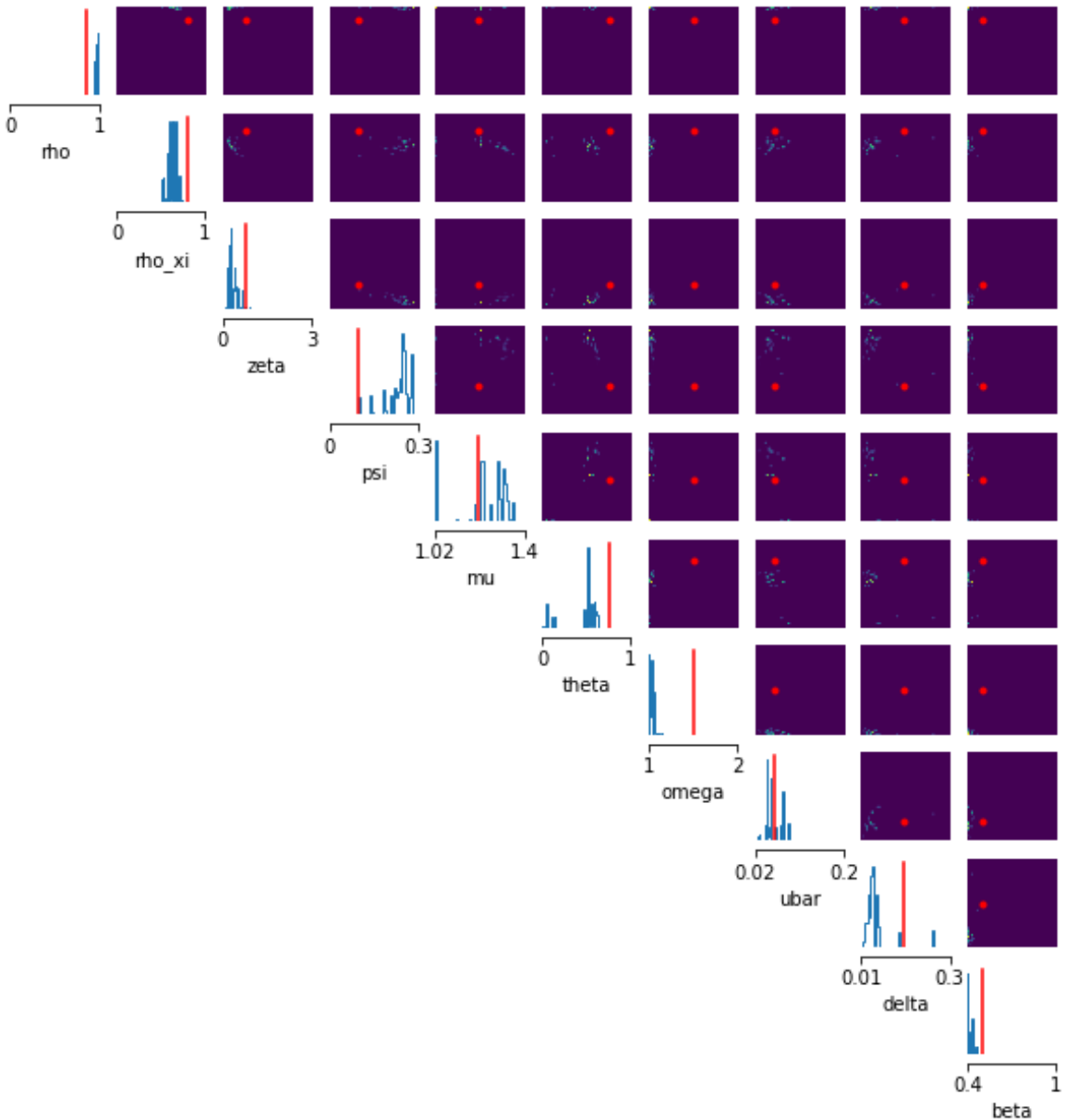
$$p* = \max_{p_t^*} \mathbb{E}_t \sum_{s=t}^{\infty} \theta^{s-t} R_{t,s}^{-1} \left[\frac{p_t^*}{P_s} y_{j,s} - w_s n_{j,s} - \psi M_s h_{j,s}\right] \tag{36}$$

$$\tag{37}$$

10 Parameter HANK Model (SNPE)



Metropolis-Hastings MCMC Estimates



Bequests Value Function Iteration Model: Setup

- Utility is a function of bequest stock and consumption

$$u(b_t, c_t) = \frac{[\phi(b_t + \underline{b})^{1-\gamma} + (1-\phi)c_t^{1-\gamma}]^{\frac{1-\rho}{1-\gamma}}}{1-\rho} \tag{38}$$

- When adjusting illiquid bequest amount, utility function is:

$$\nu(a_t, b_t) = \varphi \frac{(a_t + b_t + \underline{q})^{1-\vartheta}}{1-\vartheta} \tag{39}$$

- Value function is the max of consuming versus adjusting bequest fund

$$\begin{aligned} v_t(m_t, n_t, l_t) &= \max\{v_t^{keep}(m_t, n_t, l_t), v_t^{adj}(m_t, n_t, l_t)\} && (40) \\ &\text{s.t.} && (41) \\ x_t &= m_t + (1-\tau)n_t && (42) \end{aligned}$$

Bequests Value Function Iteration Model: Illiquid-Liquid Assets

- Value is a function of last period utility, combined with each period decision to adjust or consume goods
- Income is a function of interest on liquid assets and labor
- Bequest fund depreciates every period

$$
\begin{aligned}
w_t(a_t, b_t, l_t) &= \nu(a_t, b_t), t = T \\
w_t(a_t, b_t, l_t) &= \mathbb{E}_t \left[\max\{ v_t^{keep}(m_{t+1}, n_{t+1}, l_{t+1}), v_t^{adj}(x_{t+1}, l_{t+1}) \} \right] \\
&\text{s.t.} \\
l_{t+1} &\sim F(l_t) \\
m_{t+1} &= (1 + r_a) a_t + \omega l_{t+1} \\
n_{t+1} &= (1 - \delta) b_t \\
x_{t+1} &= m_{t+1} + (1 - \tau) n_{t+1}
\end{aligned}
$$

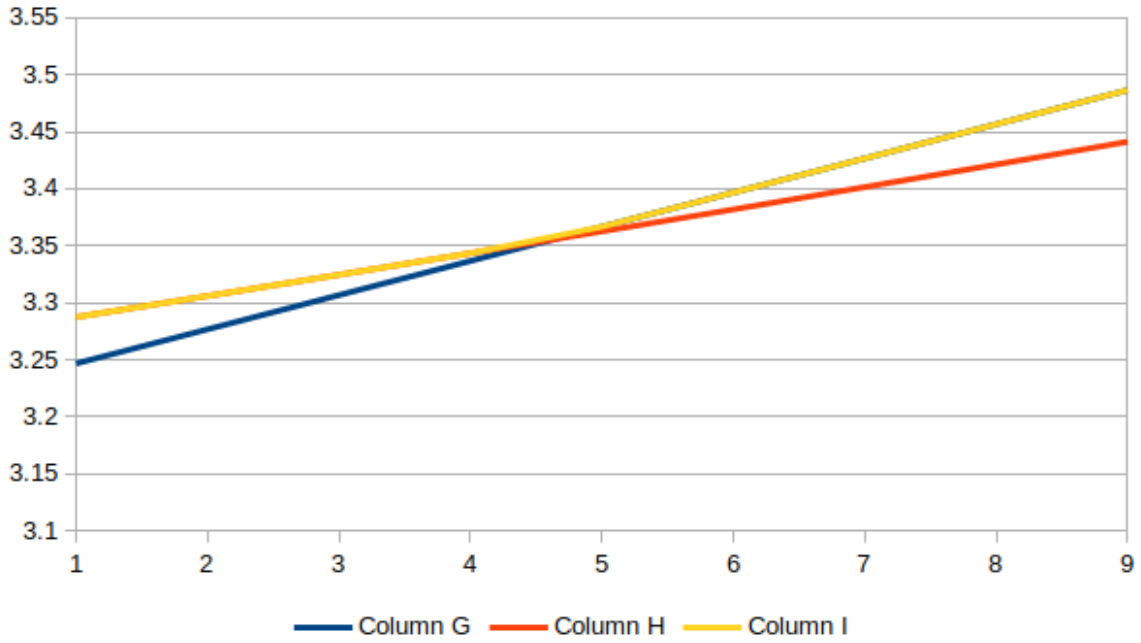
Bequests Value Function Iteration Model: Keep vs Adjust

- Keep utility function can consume, adjust bequests utility function cannot consume.

$$
\begin{aligned}
v_t^{keep}(m_t, n_t, l_t) &= \max_{c_t \in [0, m_t]} u(n_t, c_t) + \beta w_t(a_t, b_t, l_t) && (43)\\
\text{s.t.} & && (44)\\
a_t &= m_t - c_t && (45)\\
b_t &= n_t && (46)
\end{aligned}
$$

$$
\begin{aligned}
v_t^{adj}(x_t, l_t) &= \max_{b_t \in [0, x_t]} v_t^{keep}(m_t, n_t, l_t) && (47)\\
\text{s.t.} & && (48)\\
m_t &= x_t - b_t && (49)\\
n_t &= b_t && (50)
\end{aligned}
$$

Kinks in the Value Function of This Model



11 Parameter VFI model

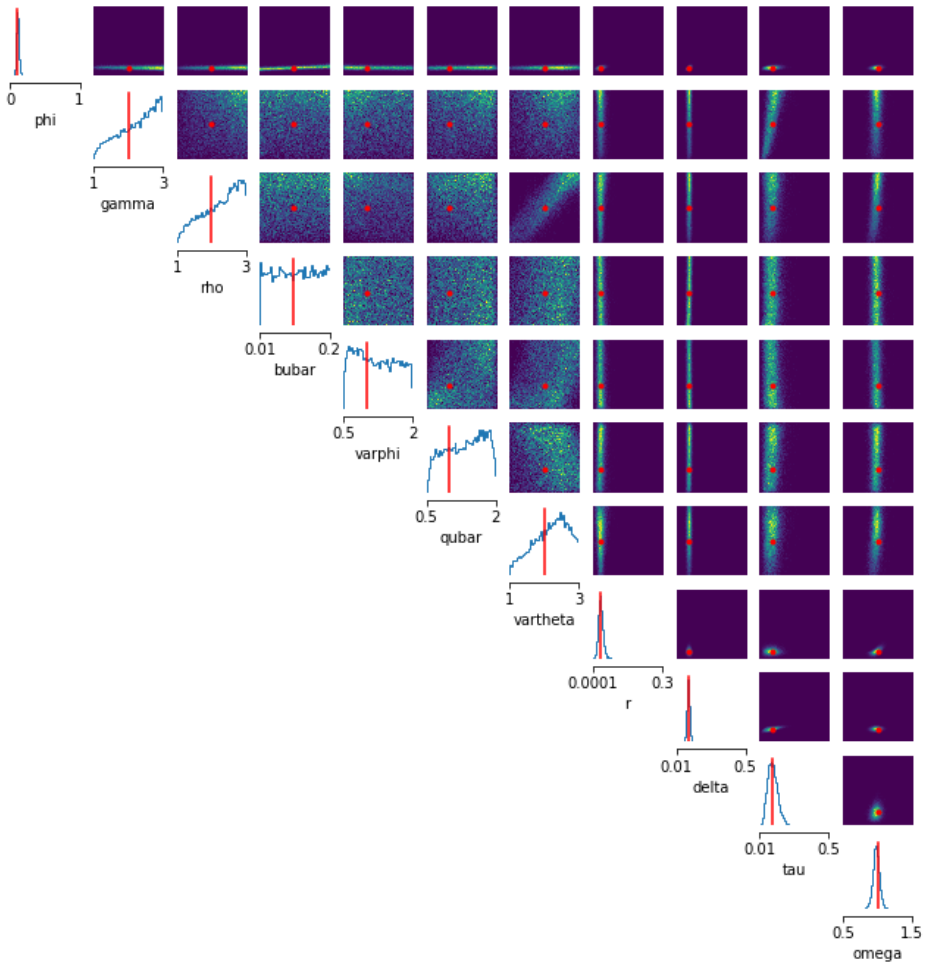


Abbildung 1: Eleven Parameter Model Estimated via VFI

Conclusion

- Extending the previous literature, I provided a method to estimate arbitrary models in a Bayesian and maximum likelihood fashion only via simulation
- I propose using the normalizing flow neural network density estimator
- I showed empirical results where the estimation procedure converges to the true parameters or reproducible posteriors

Thank You

camfen@umich.edu

Appendix

Importance Sampling

- In practice one often samples and re-estimates the posterior multiple times
- In the first round one samples $\theta \sim P(\theta)$
- But in later rounds one wants the proposal distribution for θ to be as close to the posterior as possible
- Thus in later rounds of estimation one usually samples $\theta \sim P'(\theta|X = X)$, the estimate of the posterior obtained in the preceding round
- In order to correct for sampling from the "wrong" distribution, instead of estimating just the normalizing flow density estimator $f_\phi(\theta|x)$, one estimates the density estimator along with an importance weighting factor, $f_\phi(\theta|x)\dfrac{p^{(i-1)}(\theta)}{p(\theta)}$ on samples (X, θ).

The Entire Algorithm: SNPE

Algorithm 1: SNPE Algorithm

Input: Simulator $p(x|\theta)$, prior $p(\theta)$, data x_0, flow $f_\phi(x|\theta)$, Rounds R, Samples S;

Initialize: Posterior $p^{(0)(\theta)} = p(\theta)$, data set D $= \{\}$;

for $i \leftarrow 1$ **to** R **do**

  Sample $\theta^{(n)} \sim p^{(i-1)}$ for $n = 1...S$ with Monte Carlo;

  Simulate $x^{(n)} \sim P(x|\theta^{(n)})$ for $n = 1...S$;

  Concatenate data $D = D \cup \{x^{(n)}, \theta^{(n)}\}_{n=1}^{S}$;

  while $f_\phi(x, \theta)$ *not converged* **do**

    Sample $\{x^{(i)}, \theta^{(i)}\}_i^B \sim D$ from D;

    Train $f_\phi(\theta|x)\dfrac{p^{(i-1)}(\theta)}{p(\theta)}$ on $\{x^{(i)}, \theta^{(i)}\}_i^B$;

  end

  Update posterior $p^{(i)(\theta)} = f_\phi(\theta|x_0)$;

end

Model of Firm Capital Formation

- From Toni Whited and Emil Lakkis
- Same as the RBC model, except profits after investment is the utility function

$$U(K_t, I_t, Z_t) = \sum_t \beta^t * (\pi(K_t, Z_t) - I_t) \tag{51}$$

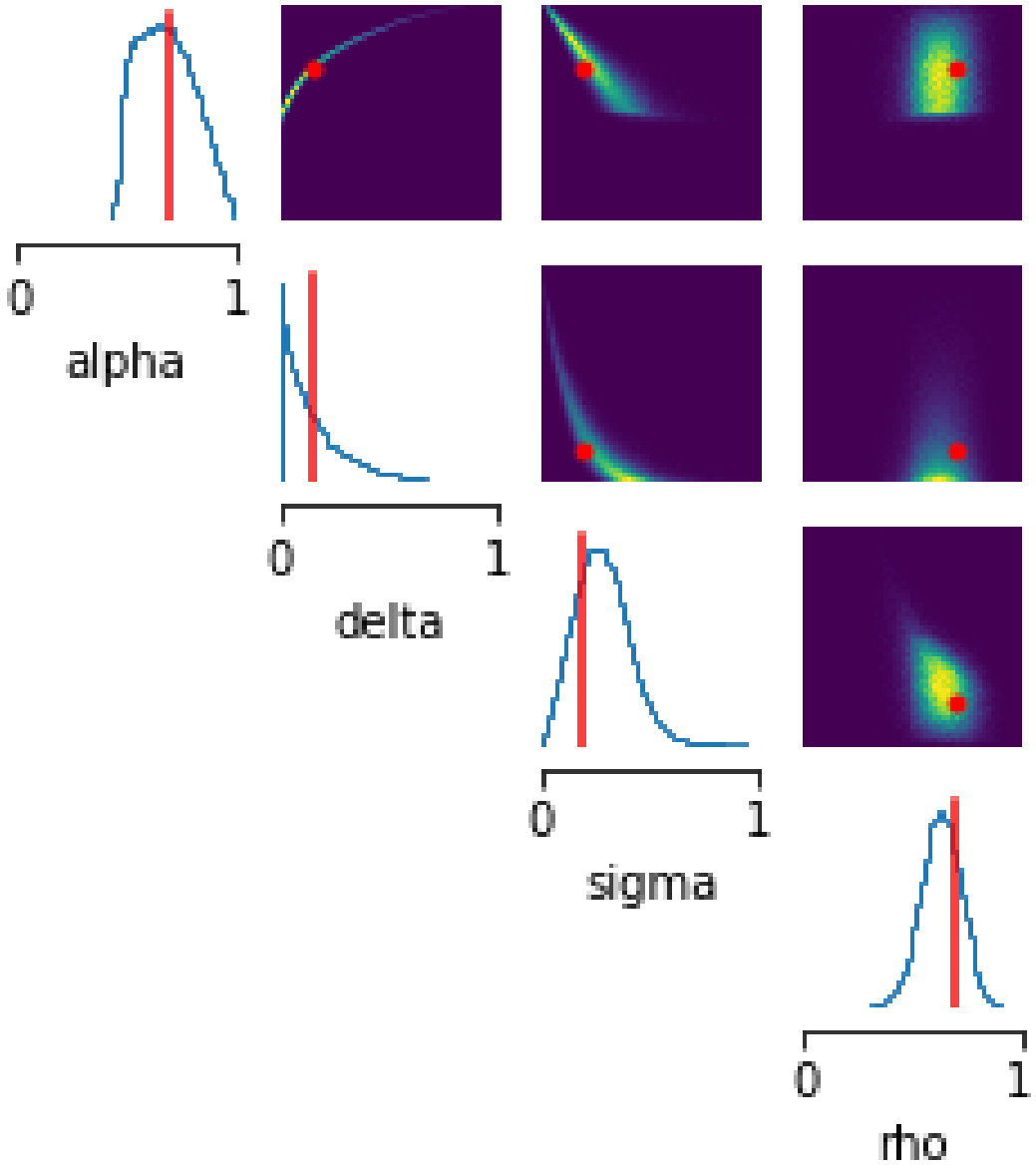
$$K_t = (1 - \delta)K_{t-1} + I_t \tag{52}$$

$$logZ_t = \rho logZ_{t-1} + \epsilon_t \tag{53}$$

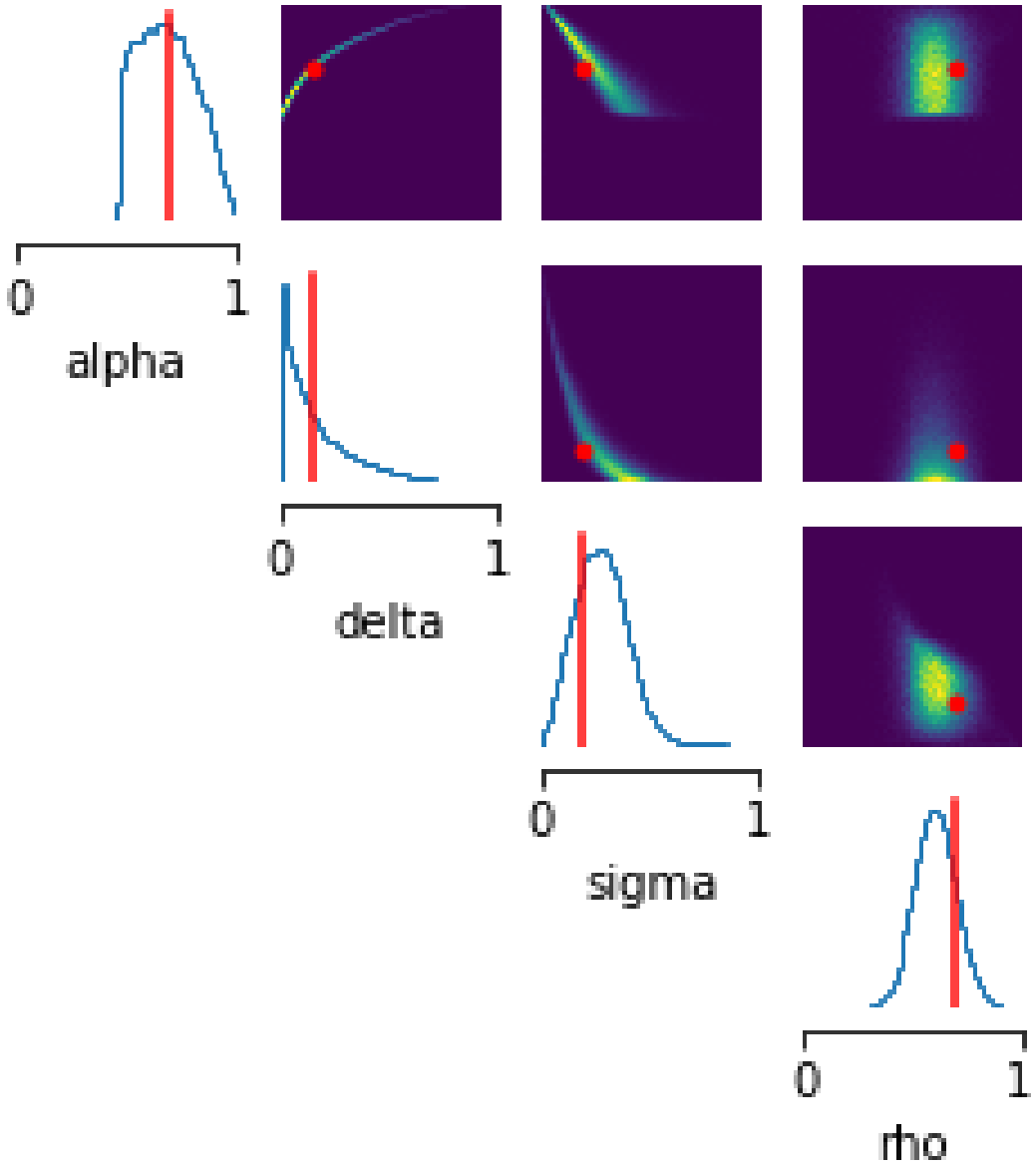
$$\pi(K_t, Z_t) = Z_t K_t^{\alpha} \tag{54}$$

$$\tag{55}$$

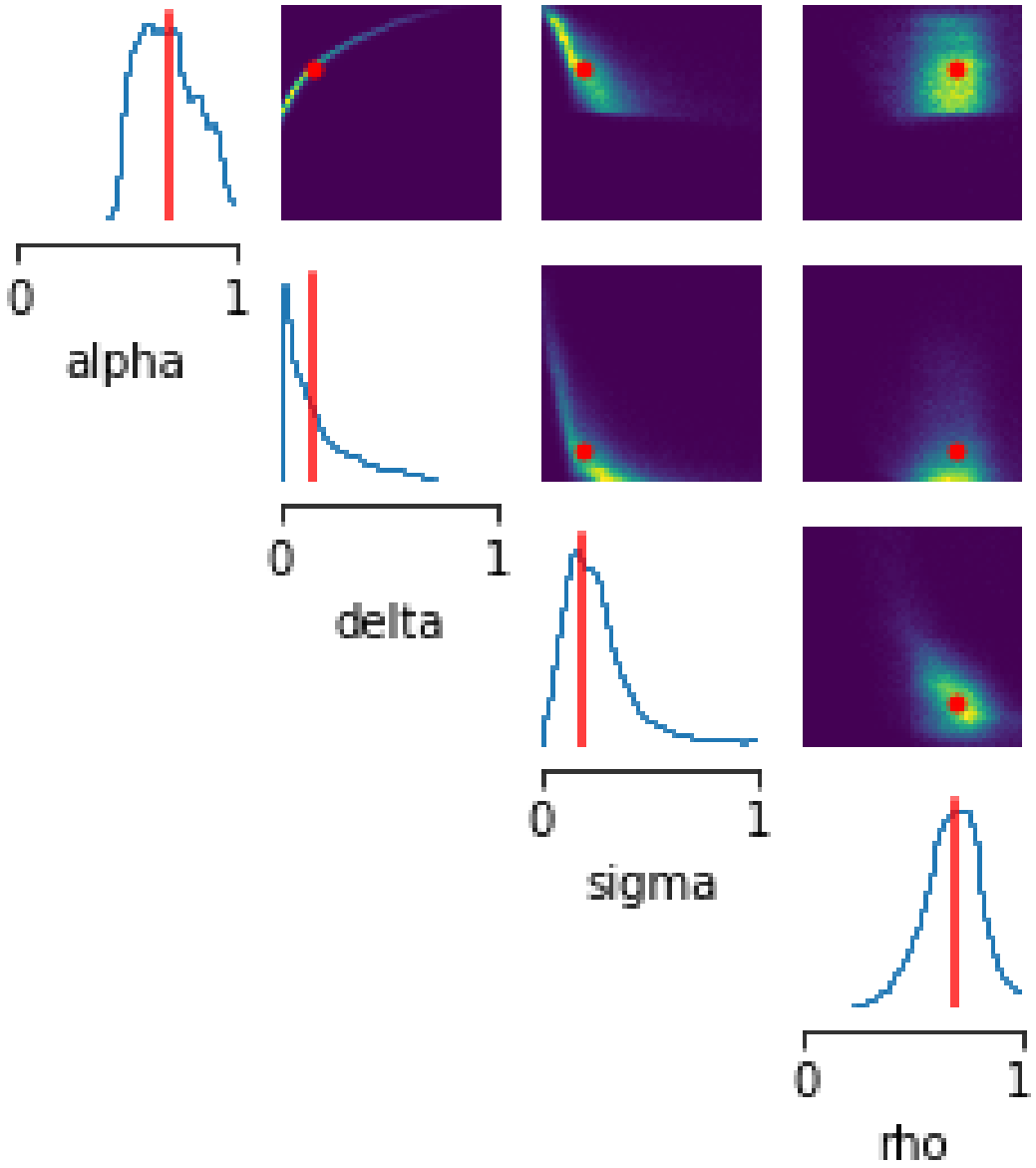
Value Function Iteration with Dense Net Embedding



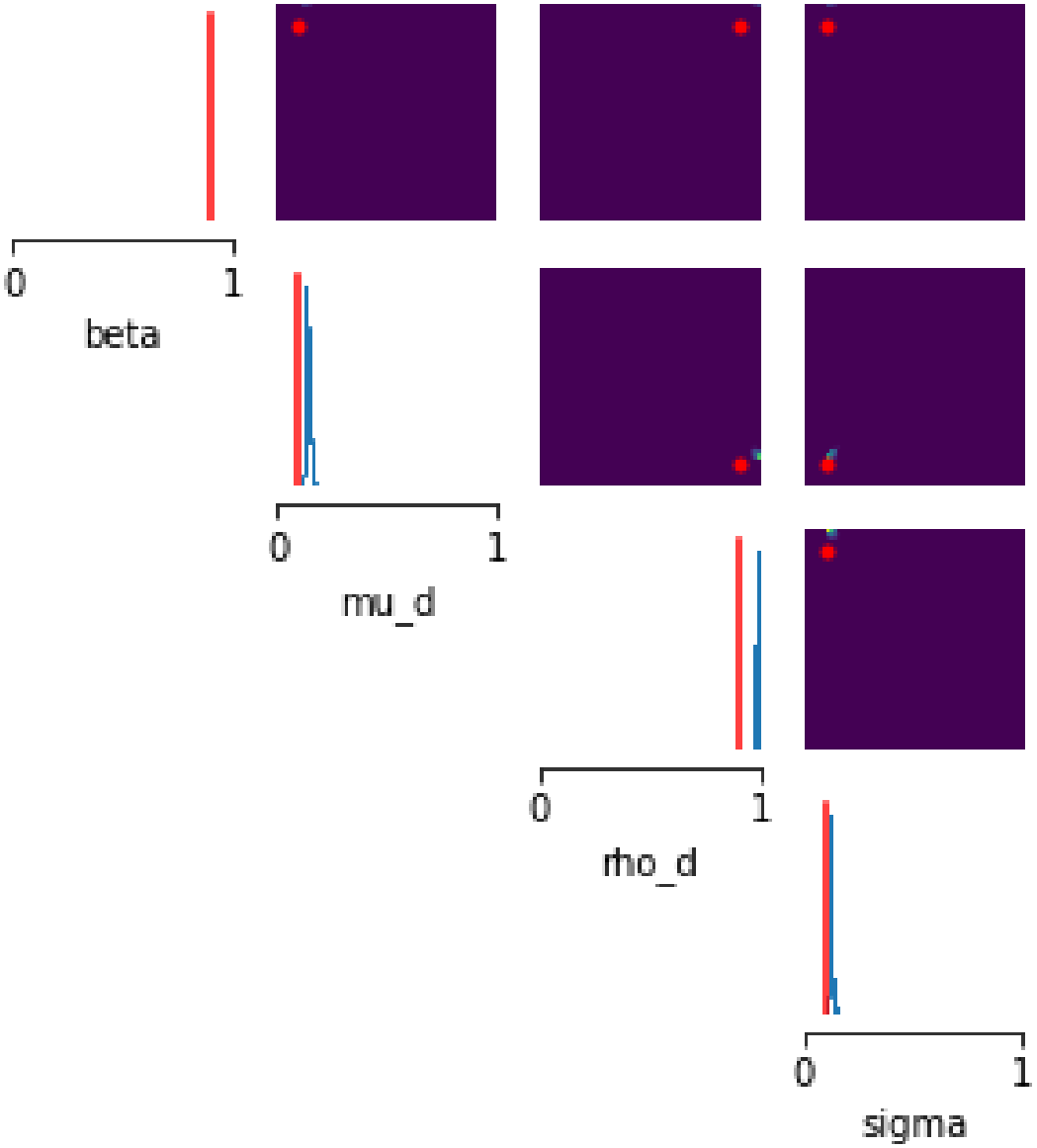
Value Function Iteration with RNN Embedding



Value Function Iteration with GAN Density Estimator



Lucas Tree Projection Model



Change of Variable Proof

Theorem (Radon Nikodym Theorem for Normalizing Flows)

1. Normalizing Flow is a Valid Change of Variable (Sketch)

- The neural autoregressive flow as set up is a bijection, if one sets the parameters of the $\alpha(.)$ and $\beta(.)$ neural networks correctly.
- As long as all the parameters are bounded, the neural autoregressive flow is Lipschitz, which implies that the input and output are absolutely continuous with respect to one another.
- Thus, one can apply the Radon-Nikodym theorem on the change in measure in both directions on the base and target distribution.
- This means the change of variable formula in the forward direction is: $P(y) = P(z)|det(df/dz)|^{-1}$, as discussed above.

Flow Universal Approximator Proof

Theorem (Flow Universal Approximator)

1. The Normalizing Flow is a Universal Approximator of Monotonic Functions (Sketch)

- First I show a step function can universally approximate any monotonic function using an argument similar to universal approximators of simple functions in measure theory.
- Then I show that a logistic regression can arbitrarily approximate any step function.
- Since $\alpha(.)$ and $\beta(.)$ are feed forward neural networks which are universal approximators of continuous functions, a neural autoregressive flow can arbitrarily approximate any logistic regression.
- Thus flows universally approximate monotonic functions

Flow Universal Approximator Proof

Theorem (Monotonic Functions and Distribution Mapping)

2. Monotonic Functions Can Universally Map Any Continuous Distribution to Any Other (Sketch)

- This proof implies that any distribution can be transformed to any other distribution via a monotonic function that is a change in variables. The proof resembles Sklar's theorem for copulas.
- First one can transform any distribution to a uniform distribution via a monotonic function by recognizing that the CDF of any distribution is exactly that transform.
- Likewise the inverse CDF allows one to transform to a uniform distribution from any other distribution via a monotonic change in variable
- Since one can map any distribution to a uniform and a uniform to any distribution, all distributions can be mapped to one another via a monotonic transform.